

Demo Overview

Demo Overview (Local Execution)

I have prepared a demo script to demonstrate **local visual execution** of the automated test flow.

Demo Coverage:

- Logging in as **Borrower**
- Navigating to **Empower Home**
- Validating:
 - Entity Information fields (Mortgage Loan)
 - Current Address field
 - Address Confidentiality Program checkboxes
 - Own/Rent checkbox
 - Loan Officer details
- Creating a new Mortgage Loan with full field validations

The local demo clearly illustrates both the **test flow** and **real-time application behavior**.

In addition, I have:

- Created an **Azure DevOps pipeline**
- Configured Node.js, command-line setup, npm installation, and report artifact generation
- Executed these test cases in CI/CD

Report Artifact: Downloadable from the provided URL.

Objectives

- Improve test coverage

- Reduce manual testing effort
 - Enable fast and reliable feedback during releases
 - Ensure faster delivery with fewer regressions
-

Automation Test Scope

We are automating **key testing areas**, including:

- **Regression Tests** – ensuring no existing functionality is broken after changes

We use **test tagging** (e.g., `@sanity`, `@regression`, `@borrower`, `@empower`) to allow selective execution.

Example Commands:

```
bash
CopyEdit
npx playwright test --grep "@sanity"
npx playwright test --grep "@borrower"
```

Benefits:

- Faster debugging during UAT
 - Flexibility in selecting which tests to run
 - Ability to scope tests per module
-

Environment Setup – Multi-Environment Testing

We have implemented **environment-based configurations** for:

- Development
- QA
- UAT
- Staging

- Production-like (for dry runs)

Each environment has a dedicated configuration file (e.g., `dev.env.ts`, `uat.env.ts`), enabling quick switching via CLI parameters or CI configuration.

Execution Time Optimization

To reduce execution time and prevent repeated logins:

1. **Authentication State Reuse** – Save authenticated session data to skip login for each test.
2. **Persistent Browser Context** – Use Playwright's persistent browser to maintain login state across test runs.

These optimizations significantly speed up the test suite and reduce flaky results.

Readable Test Cases (Gherkin Syntax)

We write scenarios using **Gherkin syntax** to make them:

- Easy to read and understand
- BDD-friendly for QA, Dev, and Product collaboration
- Directly mapped to business logic

Example:

```
gherkin
CopyEdit
Scenario: Verify borrower current address
  Given I am logged in as a borrower
  When I navigate to entity information
  Then I should see the current address field populated
```

CI/CD Integration & Reporting

Planned approach:

- Run tests in **CI/CD pipelines** for every major UAT build
- Schedule nightly regression runs
- Share **summary reports** to the Teams channel, including:
 - Status overview
 - Link to detailed report
 - Failed scenarios (if any)

This ensures **real-time visibility** and speeds up decision-making.

Client-Facing Advantages

- **Modular & scalable** test suite
- Minimal maintenance due to **robust selectors** and **shared utilities**
- Supports multiple UAT environments
- Reusable authentication mechanisms
- Transparent, review-friendly **Gherkin syntax**
- Rapid feedback after deployments
- Easy expansion to additional modules